UNIVERSITÉ HASSAN II DE CASABLANCA

FACULTÉ DES SCIENCES BEN M'SICK

Département Informatique

Master d'Excellence en Intelligence Artificielle

Mise en place d'une infrastructure DevOps avec CI/CD, Kubernetes

---

Encadré par :
Pr. F. Benabbou

Réalisé par :
Rami Rajae
Bousnitra Douaa

Année Universitaire 2025–2026

Table des matières

1 Architecture du projet **3**
1.1 Présentation du concept DevOps . . . 3
1.2 Architecture globale du projet . . . 3
1.3 Description des composants . . . 4
1.4 Schéma d'architecture . . . 4
1.5 Flux de fonctionnement . . . 5

2 Mise en place de l'infrastructure cloud avec Terraform **6**
2.1 Présentation de Terraform . . . 6
2.2 Architecture cloud (AWS) . . . 6
2.3 Création des ressources AWS . . . 6
2.4 Organisation des fichiers Terraform . . . 7
2.5 Commandes d'exécution . . . 8
2.6 Résultat de l'infrastructure . . . 8

3 Configuration des serveurs avec Ansible **9**
3.1 Présentation d'Ansible . . . 9
3.2 Installation des dépendances . . . 9
3.3 Configuration du Master Node . . . 10
3.4 Configuration du Worker Node . . . 10
3.5 Vérification du cluster Kubernetes . . . 11

4 Développement de l'application **12**
4.1 Présentation de l'application . . . 12
4.2 Choix technologique (Flask) . . . 12
4.3 Architecture de l'application . . . 13
4.4 Endpoints de l'API . . . 13
4.5 Tests locaux de l'application . . . 14

5 Conteneurisation avec Docker **15**
5.1 Présentation de l'application . . . 15
5.2 Développement avec Flask . . . 15
5.3 Création du Dockerfile . . . 15
5.4 Build et push de l'image Docker . . . 16

6 Déploiement avec Kubernetes **17**
6.1 Présentation de Kubernetes 17
6.2 Déploiement (Deployment) 17
6.3 Service et exposition de l'application 18
6.4 Accès à l'application 19

7 Pipeline CI/CD avec GitHub Actions **20**
7.1 Présentation de GitHub Actions 20
7.2 Étapes du pipeline 20
7.3 Tests, build et déploiement automatique 21
7.4 Résultat du pipeline 21

8 Résultats **22**
8.1 Résultats obtenus et vérification du système 22

Table des figures

1.4.1 Schéma d'architecture 4

2.4.1 Le fichier variables.tf 7
2.4.2 Le fichier main.tf 7

3.2.1 Le fichier inventory.tf 9
3.2.2 Le fichier playbook.tf 10

4.3.1 Le fichier app.py 13
4.4.1 Le fichier test_app.py 13

5.3.1 Le fichier requirement.txt 15
5.3.2 Le fichier DockerFile 16

6.2.1 Le fichier Deployement.yaml 17
6.3.1 Le fichier Service.yaml 18

7.2.1 Le fichier CI-CD.yaml 20

Introduction

Le DevOps est une approche moderne de développement logiciel qui vise à améliorer la collaboration entre les équipes de développement (Dev) et les équipes d'exploitation ou d'infrastructure (Ops). Traditionnellement, ces deux équipes travaillaient de manière séparée : les développeurs écrivaient le code, puis l'équipe d'exploitation était responsable du déploiement et de la gestion des serveurs. Cette séparation pouvait provoquer des problèmes de communication, des erreurs lors du déploiement et des délais importants pour mettre une application en production.

L'approche DevOps cherche à résoudre ces problèmes en introduisant l'automatisation, l'intégration continue et le déploiement continu. L'objectif est de créer un processus fluide dans lequel le code peut être testé, construit et déployé automatiquement dès qu'une modification est apportée. Cela permet de réduire les erreurs humaines, d'améliorer la qualité du logiciel et d'accélérer la livraison des nouvelles fonctionnalités.

Dans le cadre de ce projet, nous avons mis en place une chaîne DevOps complète permettant de déployer automatiquement une application conteneurisée sur un cluster Kubernetes hébergé sur le cloud. Plusieurs outils DevOps ont été utilisés afin de couvrir toutes les étapes du cycle de vie de l'application. Terraform a été utilisé pour créer l'infrastructure cloud, Ansible pour configurer les serveurs, Docker pour conteneuriser l'application, Kubernetes pour orchestrer les conteneurs, et GitHub Actions pour automatiser le pipeline CI/CD. L'ensemble de l'infrastructure est hébergé sur Amazon Web Services (AWS).

L'objectif principal de ce projet est de démontrer comment ces technologies peuvent être combinées afin de créer une infrastructure moderne, automatisée et facilement reproductible.

Chapitre 1

Architecture du projet

1.1 Présentation du concept DevOps

Le DevOps est une approche méthodologique qui vise à unifier les processus de développement logiciel (Development) et d'exploitation des systèmes (Operations). L'objectif principal est de réduire le temps de mise en production tout en améliorant la qualité, la fiabilité et la sécurité des applications. Cette approche repose sur plusieurs pratiques clés :

- **Intégration Continue (CI) :** consiste à intégrer régulièrement le code dans un dépôt commun afin de détecter rapidement les erreurs.
- **Déploiement Continu (CD) :** permet d'automatiser le déploiement des applications après validation.
- **Infrastructure as Code (IaC) :** gestion de l'infrastructure via du code avec des outils comme Terraform.
- **Automatisation :** réduction des interventions manuelles grâce à des outils comme Ansible
- **Conteneurisation :** utilisation de Docker pour standardiser les environnements d'exécution.
- **Orchestration :** gestion des conteneurs avec Kubernetes.

Dans ce projet, ces principes sont appliqués pour automatiser l'ensemble du cycle de vie de l'application, depuis son développement jusqu'à son déploiement sur le cloud

1.2 Architecture globale du projet

Avant l'implémentation technique, une architecture globale du système a été conçue à l'aide de l'outil Draw.io afin de représenter visuellement les différents composants ainsi que leurs interactions. Cette architecture est organisée en deux environnements distincts. D'une part, un environnement local qui comprend le développeur ainsi que les outils d'automatisation. Cet environnement permet le développement de l'application, la gestion du code source et le déclenchement des pipelines d'intégration et de déploiement continus. D'autre part, un environnement cloud hébergé sur Amazon Web Services, qui regroupe l'infrastructure virtuelle, notamment les instances EC2, le cluster Kubernetes ainsi que l'exécution de l'application conteneurisée. Cette séparation entre les deux environnements

permet d'assurer une meilleure organisation du système, de renforcer la sécurité et de garantir une scalabilité efficace du déploiement.

1.3 Description des composants

L'architecture du système repose sur plusieurs composants essentiels qui interagissent afin d'assurer un processus de déploiement automatisé. Le développeur constitue le point de départ du système, en étant responsable de l'écriture et de la modification du code sur son environnement local. Une fois le code prêt, il est envoyé vers un dépôt distant hébergé sur GitHub, qui assure la gestion de versions et le stockage du code source. Ce dépôt permet également de déclencher automatiquement des processus d'intégration continue via GitHub Actions, qui exécute les différentes étapes du pipeline CI/CD telles que les tests, la construction de l'image et le déploiement. La conteneurisation de l'application est assurée par Docker, qui permet de créer des environnements d'exécution portables et cohérents. Les images générées sont ensuite stockées sur Docker Hub, facilitant ainsi leur distribution et leur utilisation lors du déploiement. Par ailleurs, l'infrastructure cloud est provisionnée automatiquement grâce à Terraform, qui permet de définir et de créer les ressources nécessaires sur Amazon Web Services, telles que les machines virtuelles et les configurations réseau. Une fois l'infrastructure mise en place, Ansible est utilisé pour configurer les serveurs de manière automatique, notamment pour installer Docker et Kubernetes et préparer l'environnement d'exécution. Enfin, l'orchestration des conteneurs est assurée par Kubernetes, qui gère le déploiement, la scalabilité et la disponibilité de l'application au sein du cluster. L'ensemble de ces composants fonctionne de manière coordonnée pour garantir un système automatisé, fiable et évolutif.

1.4 Schéma d'architecture

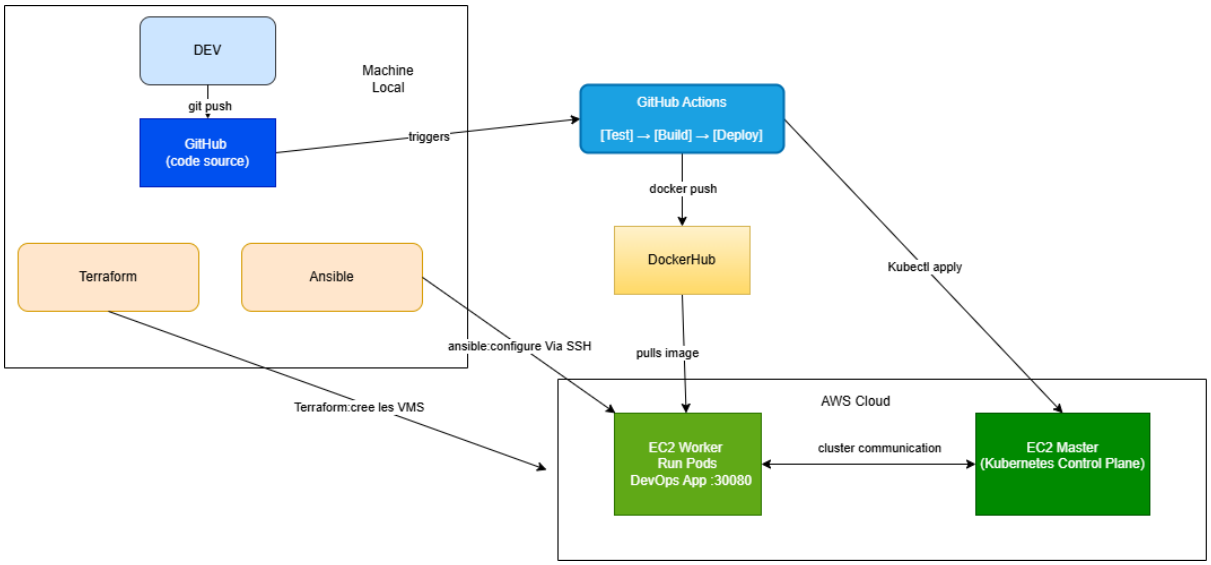


FIGURE 1.4.1 – Schéma d'architecture

Le schéma d'architecture, réalisé avec Draw.io, représente visuellement l'ensemble du système. Il met en évidence les interactions entre les différents composants ainsi que le flux de données. Il montre notamment :

- Le transfert du code depuis le développeur vers GitHub
- Le déclenchement automatique du pipeline CI/CD
- La création et le stockage des images Docker dans Docker Hub
- Le déploiement de l'application sur le cluster Kubernetes hébergé sur Amazon Web Services

1.5 Flux de fonctionnement

Le fonctionnement global du système repose sur une chaîne d'automatisation continue reliant les différentes étapes du développement et du déploiement. Dans un premier temps, le développeur écrit le code de l'application sur son environnement local, puis l'envoie vers le dépôt distant sur GitHub à l'aide d'une commande git push. Cette action déclenche automatiquement un pipeline d'intégration et de déploiement continus via GitHub Actions. Le pipeline commence par exécuter les tests afin de vérifier la validité du code, puis procède à la construction d'une image à l'aide de Docker. Une fois l'image générée, elle est envoyée vers Docker Hub, où elle est stockée et rendue accessible pour le déploiement. Parallèlement, Terraform est utilisé pour créer et gérer l'infrastructure sur Amazon Web Services, tandis que Ansible configure automatiquement les serveurs en installant les outils nécessaires. Ensuite, le pipeline exécute la commande de déploiement sur Kubernetes, permettant de lancer l'application sous forme de conteneurs au sein du cluster. Les conteneurs sont exécutés sur le nœud Worker, tandis que le nœud Master assure la gestion et la coordination du cluster. Ainsi, l'ensemble du processus, depuis le développement jusqu'au déploiement, est entièrement automatisé, ce qui garantit une exécution rapide, fiable et sans intervention manuelle.

Chapitre 2

Mise en place de l'infrastructure cloud avec Terraform

2.1 Présentation de Terraform

Terraform est un outil d'Infrastructure as Code (IaC) qui permet de définir, créer et gérer des ressources informatiques à l'aide de fichiers de configuration. Contrairement à une gestion manuelle via des interfaces graphiques, Terraform offre la possibilité d'automatiser entièrement le déploiement de l'infrastructure. Cette approche permet de décrire les ressources sous forme de code, ce qui facilite leur gestion, leur modification et leur suppression. Elle présente également l'avantage de rendre l'infrastructure reproductible et versionnable, au même titre que le code source d'une application, tout en réduisant les risques d'erreurs humaines.

2.2 Architecture cloud (AWS)

L'infrastructure du projet est déployée sur Amazon Web Services, qui fournit un ensemble de services cloud permettant de créer des environnements flexibles et évolutifs. L'architecture mise en place repose sur la création d'un réseau virtuel privé, ainsi que sur des mécanismes de sécurité et de connectivité. Elle inclut également deux instances EC2 qui forment la base d'un cluster Kubernetes, composé d'un nœud Master responsable de la gestion du cluster et d'un nœud Worker chargé d'exécuter les conteneurs de l'application. Cette architecture permet d'assurer une bonne organisation des ressources ainsi qu'une séparation claire des responsabilités.

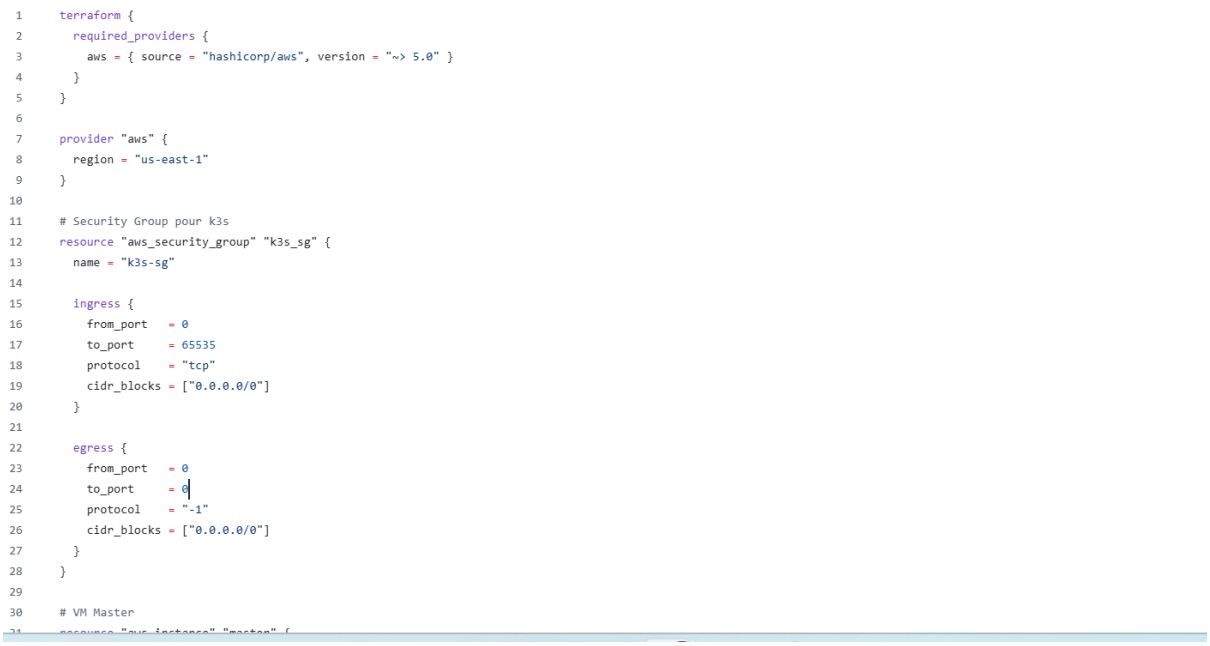
2.3 Création des ressources AWS

Dans ce projet, plusieurs ressources ont été créées à l'aide de Terraform afin de constituer une infrastructure complète et fonctionnelle. Tout d'abord, un VPC (Virtual Private Cloud) a été mis en place afin de créer un réseau virtuel isolé dans lequel les ressources peuvent être déployées en toute sécurité. À l'intérieur de ce réseau, un subnet public a été configuré afin de permettre aux instances de disposer d'adresses IP publiques et d'être accessibles depuis Internet. Afin d'assurer la communication entre le réseau interne

et l'extérieur, une Internet Gateway a été ajoutée. Cette dernière permet aux instances situées dans le VPC d'accéder à Internet. Une table de routage a également été définie et associée au subnet afin de diriger correctement le trafic réseau vers cette passerelle. Par ailleurs, un Security Group a été configuré pour contrôler les accès aux instances en jouant le rôle de pare-feu. Les ports nécessaires, notamment pour SSH, HTTP et Kubernetes, ont été autorisés. Enfin, deux instances EC2 ont été créées automatiquement. L'une est utilisée comme nœud Master pour gérer le cluster Kubernetes, tandis que l'autre sert de nœud Worker pour exécuter les conteneurs de l'application. L'ensemble de ces ressources constitue l'infrastructure nécessaire au bon fonctionnement du système.

2.4 Organisation des fichiers Terraform

FIGURE 2.4.1 – Le fichier variables.tf



```
1  terraform {
2    required_providers {
3      aws = { source = "hashicorp/aws", version = "~> 5.0" }
4    }
5  }
6
7  provider "aws" {
8    region = "us-east-1"
9  }
10
11 # Security Group pour k3s
12 resource "aws_security_group" "k3s_sg" {
13   name = "k3s-sg"
14
15   ingress {
16     from_port   = 0
17     to_port     = 65535
18     protocol    = "tcp"
19     cidr_blocks = ["0.0.0.0/0"]
20   }
21
22   egress {
23     from_port   = 0
24     to_port     = 0
25     protocol    = "-1"
26     cidr_blocks = ["0.0.0.0/0"]
27   }
28 }
29
30 # VM Master
```

FIGURE 2.4.2 – Le fichier main.tf

Le projet Terraform est structuré en plusieurs fichiers de configuration afin de faciliter son organisation et sa maintenance. Le fichier variables.tf permet de définir les différentes

variables utilisées dans le projet, ce qui rend la configuration plus flexible et réutilisable. Le fichier main.tf contient la définition des ressources à créer sur le cloud.Cette organisation modulaire permet de rendre le code plus lisible, plus clair et plus facile à maintenir

2.5 Commandes d'exécution

Le déploiement de l'infrastructure à l'aide de Terraform repose sur l'exécution de plusieurs commandes essentielles. La commande **terraform init** permet d'initialiser le projet en téléchargeant les dépendances nécessaires. Ensuite, la commande **terraform validate** est utilisée pour vérifier la syntaxe des fichiers de configuration. La commande **terraform plan** permet de visualiser les modifications qui seront apportées à l'infrastructure avant leur application. Enfin, la commande **terraform apply** exécute réellement les changements et crée les ressources sur le cloud. Ces différentes étapes assurent un déploiement contrôlé et sécurisé de l'infrastructure.

2.6 Résultat de l'infrastructure

À l'issue de l'exécution des commandes Terraform, l'ensemble de l'infrastructure est automatiquement déployé sur Amazon Web Services. Les instances EC2 sont opérationnelles et configurées pour accueillir les étapes suivantes du projet, notamment l'installation et la configuration du cluster Kubernetes. Cette approche permet de créer une infrastructure complète en quelques minutes seulement, de manière fiable, reproductible et sans intervention manuelle. Elle constitue ainsi un élément clé dans la mise en œuvre d'une démarche DevOps moderne

Chapitre 3

Configuration des serveurs avec Ansible

3.1 Présentation d'Ansible

Une fois les machines virtuelles créées, celles-ci sont initialement vides et ne contiennent pas les logiciels nécessaires au fonctionnement du projet. Afin d'automatiser l'installation et la configuration des différents composants sur ces machines, l'outil Ansible a été utilisé. Ansible est un outil d'automatisation open source permettant de gérer et configurer plusieurs serveurs à distance à partir d'une machine de contrôle. Il repose sur une approche agentless, ce qui signifie qu'aucun agent n'est requis sur les machines distantes. Les tâches à exécuter sont décrites dans des fichiers appelés playbooks, écrits en YAML, qui définissent de manière déclarative les opérations à réaliser sur les serveurs.

3.2 Installation des dépendances

FIGURE 3.2.1 – Le fichier inventory.tf

```yaml
- name: Installer Docker sur tous les noeuds
  hosts: all
  become: yes
  tasks:
    - name: Update yum
      yum:
        name: "*"
        state: latest

    - name: Installer Docker
      yum:
        name: docker
        state: present

    - name: Démarrer Docker
      service:
        name: docker
        state: started
        enabled: yes

- name: Installer k3s sur le master
  hosts: master
  become: yes
  tasks:
```

FIGURE 3.2.2 – Le fichier playbook.tf

Avant d'utiliser Ansible, il est nécessaire de configurer la machine de contrôle ainsi que les accès aux machines distantes. Cela inclut notamment l'installation de Python et d'Ansible sur la machine locale. Par ailleurs, une configuration des clés SSH est requise afin d'assurer une communication sécurisée entre la machine de contrôle et les instances distantes. Un inventaire Ansible est également mis en place afin de recenser les machines cibles du projet. Cet inventaire contient les adresses IP des instances EC2 correspondant au Master Node et au Worker Node, permettant ainsi à Ansible d'exécuter les tâches définies dans les playbooks sur les bons serveurs.

3.3 Configuration du Master Node

La configuration du Master Node constitue une étape essentielle dans la mise en place du cluster Kubernetes. Dans ce projet, un playbook Ansible a été utilisé pour automatiser l'installation des composants nécessaires. Ce playbook permet tout d'abord d'installer Docker, qui est indispensable pour l'exécution des conteneurs. Ensuite, Kubernetes est installé sur la machine, notamment les outils kubeadm, kubelet et kubectl. Une fois l'installation terminée, le cluster est initialisé à l'aide de la commande kubeadm init. Cette initialisation permet de configurer le Master Node comme point central du cluster et génère une commande de jointure qui sera utilisée par les autres machines pour rejoindre le cluster.

3.4 Configuration du Worker Node

Après la configuration du Master Node, le Worker Node est préparé afin de rejoindre le cluster Kubernetes. Cette étape est également automatisée à l'aide d'un playbook Ansible. Le playbook installe les mêmes dépendances que sur le Master Node, notamment Docker et les composants Kubernetes nécessaires au fonctionnement du nœud. Ensuite, le Worker Node rejoint le cluster en exécutant la commande de jointure générée lors de

l'initialisation du Master Node via kubeadm init. Une fois cette opération effectuée, le Worker Node devient un membre actif du cluster et est capable d'exécuter des pods et des conteneurs, permettant ainsi de répartir les charges de travail.

3.5 Vérification du cluster Kubernetes

Une fois la configuration terminée, il est important de vérifier que le cluster Kubernetes fonctionne correctement. Cette vérification est réalisée depuis le Master Node à l'aide de la commande kubectl. La commande kubectl get nodes permet d'afficher la liste des machines appartenant au cluster ainsi que leur état. Si la configuration est correcte, les nœuds doivent apparaître avec le statut "Ready", indiquant qu'ils sont opérationnels et prêts à exécuter des charges de travail. Cette étape confirme le bon fonctionnement de l'ensemble de l'infrastructure mise en place et valide la configuration réalisée à l'aide d'Ansible.

Chapitre 4

Développement de l'application

4.1 Présentation de l'application

Pour démontrer le fonctionnement de l'infrastructure DevOps mise en place dans ce projet, une application simple a été développée. Cette application a pour objectif principal de tester l'ensemble de la chaîne de déploiement, depuis la création de l'infrastructure jusqu'à l'exécution de l'application dans un cluster Kubernetes. L'application est conçue comme un service web exposant une API REST capable de retourner des données au format JSON. Elle permet ainsi de valider le bon fonctionnement des services déployés ainsi que la communication entre les différents composants de l'architecture.

4.2 Choix technologique (Flask)

Le framework Flask a été choisi pour le développement de cette application. Flask est un micro-framework web léger développé en Python, reconnu pour sa simplicité, sa flexibilité et sa facilité d'utilisation. Il permet de créer rapidement des applications web ainsi que des API REST sans imposer une structure rigide. Ce choix est particulièrement adapté dans un contexte DevOps, où l'objectif est de mettre en place rapidement une application simple permettant de tester l'infrastructure et les mécanismes de déploiement. Flask offre également une bonne compatibilité avec les conteneurs Docker, ce qui facilite son intégration dans un environnement Kubernetes

4.3 Architecture de l'application

Code Blame 13 lines (10 loc) · 298 Bytes

```python
from flask import Flask, jsonify
app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({'message': 'DevOps App DSBD', 'status': 'okk'})

@app.route('/health')
def health():
    return jsonify({'status': 'ok'}), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

FIGURE 4.3.1 – Le fichier app.py

L'architecture de l'application est volontairement simple afin de se concentrer sur l'intégration avec l'infrastructure DevOps. Elle repose sur une application monolithique développée en Flask, exposant plusieurs routes accessibles via HTTP. L'application est conteneurisée à l'aide de Docker, ce qui permet de l'exécuter de manière isolée dans un environnement standardisé. Une fois conteneurisée, l'application est déployée sur le cluster Kubernetes, où elle est exécutée sous forme de pod. L'architecture globale s'intègre dans le pipeline DevOps mis en place, où l'application est développée, conteneurisée, puis déployée automatiquement sur l'infrastructure cloud provisionnée avec Terraform et configurée avec Ansible.

4.4 Endpoints de l'API

Code Blame 9 lines (7 loc) · 198 Bytes

```python
from app import app

def test_home():
    c = app.test_client()
    assert c.get('/').status_code == 200

def test_health():
    c = app.test_client()
    assert c.get('/health').status_code == 200
```

FIGURE 4.4.1 – Le fichier test_app.py

L'application expose une API REST simple composée de plusieurs endpoints permettant d'interagir avec le service. La route principale (/) retourne un message JSON indiquant que l'application fonctionne correctement. Cette route permet de vérifier rapidement que le service est accessible et opérationnel. Une seconde route /health a été implémentée afin de fournir un mécanisme de vérification de l'état de santé de l'application. Cette route est utilisée par Kubernetes dans le cadre des health checks. Elle permet de déterminer si l'application est toujours fonctionnelle. En cas de réponse incorrecte ou d'absence de réponse, Kubernetes peut automatiquement redémarrer le conteneur afin de maintenir la disponibilité du service. Ce mécanisme est essentiel dans les environnements de production, car il garantit la résilience et la continuité de service des applications déployées.

4.5 Tests locaux de l'application

Avant son déploiement sur le cluster Kubernetes, l'application a été testée localement afin de vérifier son bon fonctionnement. Ces tests consistent à exécuter l'application en local et à envoyer des requêtes HTTP aux différents endpoints exposés. L'objectif de cette étape est de s'assurer que l'application répond correctement, que les routes sont accessibles et que les réponses JSON sont conformes aux attentes. Une fois les tests validés, l'application est prête à être conteneurisée et déployée dans l'environnement Kubernetes. Ces tests locaux permettent de détecter et corriger les éventuelles erreurs avant le déploiement, réduisant ainsi les risques d'échec lors de l'intégration dans l'infrastructure globale.

Chapitre 5

Conteneurisation avec Docker

5.1 Présentation de l'application

Dans le cadre de ce projet DevOps, une application simple a été développée afin de démontrer le bon fonctionnement de l'ensemble de la chaîne de déploiement. Cette application sert principalement à valider l'infrastructure mise en place ainsi que les mécanismes de conteneurisation et d'orchestration. L'application est conçue comme un service web léger exposant une API REST capable de retourner des réponses au format JSON. Elle permet de tester la communication entre les différents composants du système, notamment les conteneurs Docker et le cluster Kubernetes.

5.2 Développement avec Flask

L'application a été développée en utilisant le framework Flask, un micro-framework web basé sur le langage Python. Flask est reconnu pour sa simplicité, sa légèreté et sa flexibilité, ce qui en fait un choix adapté pour le développement d'applications web et d'API REST dans un contexte de projet DevOps. Le développement avec Flask consiste à définir des routes HTTP qui répondent à différentes requêtes clients. Chaque route est associée à une fonction Python qui retourne une réponse, généralement au format JSON. Cette approche permet de créer rapidement des services web simples tout en conservant une structure claire et modulaire

5.3 Création du Dockerfile

Code Blame 3 lines (3 loc) · 44 Bytes

```
1  flask==3.0.0
2  pytest==7.4.0
3  requests==2.31.0
```

FIGURE 5.3.1 – Le fichier requirement.txt

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Run with Gunicorn for production
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app:app"]
```

FIGURE 5.3.2 – Le fichier DockerFile

Après le développement de l'application, celle-ci a été conteneurisée à l'aide de Docker. La conteneurisation consiste à regrouper l'application ainsi que toutes ses dépendances dans une image portable pouvant être exécutée sur n'importe quel environnement disposant de Docker. Pour cela, un fichier Dockerfile a été créé. Ce fichier contient l'ensemble des instructions nécessaires à la construction de l'image Docker de l'application. Tout d'abord, une image de base légère basée sur Python est utilisée. Ensuite, les dépendances nécessaires sont installées à partir du fichier requirements.txt. Le code source de l'application est ensuite copié dans l'image, puis le port utilisé par l'application est exposé afin de permettre son accès depuis l'extérieur du conteneur. Enfin, une commande de démarrage est définie pour lancer l'application lors de l'exécution du conteneur.

5.4 Build et push de l'image Docker

Une fois le Dockerfile configuré, l'image Docker de l'application est construite localement à l'aide de la commande docker build. Cette étape permet de générer une image contenant l'ensemble de l'application prête à être exécutée dans un conteneur. Après la construction, il est possible de tester l'image en la lançant localement dans un conteneur Docker afin de vérifier le bon fonctionnement de l'application. Une fois les tests validés, l'image Docker est publiée sur DockerHub, qui est un registre public d'images Docker. Cette étape consiste à taguer l'image puis à la pousser vers le registre à l'aide de la commande docker push. Le stockage de l'image dans DockerHub permet de la rendre accessible depuis Internet, ce qui est essentiel pour le déploiement sur Kubernetes. En effet, le cluster peut récupérer automatiquement l'image depuis ce registre afin de créer les conteneurs nécessaires à l'exécution de l'application.

Chapitre 6

Déploiement avec Kubernetes

6.1 Présentation de Kubernetes

Kubernetes est une plateforme open source d'orchestration de conteneurs utilisée pour automatiser le déploiement, la gestion et la mise à l'échelle des applications conteneurisées. Dans ce projet, Kubernetes est utilisé pour gérer l'exécution des conteneurs Docker de l'application développée. Kubernetes permet de garantir la disponibilité et la résilience des applications en assurant leur redémarrage automatique en cas de panne, ainsi que leur répartition sur différents nœuds du cluster. Il offre également des mécanismes avancés de gestion des ressources, de mise à l'échelle et de supervision, ce qui en fait un outil essentiel dans une architecture DevOps moderne.

6.2 Déploiement (Deployment)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: devops-app
  template:
    metadata:
      labels:
        app: devops-app
    spec:
      containers:
        - name: devops-app
          image: douaabousnitra/devops-app:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 5000
```

FIGURE 6.2.1 – Le fichier Deployement.yaml

Le déploiement de l'application dans Kubernetes se fait à l'aide de fichiers de configuration au format YAML, appelés manifests. Ces fichiers permettent de décrire de manière déclarative l'état souhaité des ressources au sein du cluster. Dans un premier temps, un namespace est créé afin d'isoler les ressources du projet des autres ressources présentes dans le cluster. Cette approche permet d'organiser et de structurer les différents composants de manière logique. Ensuite, un objet de type Deployment est défini pour déployer l'application. Le Deployment spécifie plusieurs paramètres importants, notamment l'image Docker à utiliser, le nombre de réplicas ainsi que les règles de redémarrage des pods. Dans ce projet, deux réplicas de l'application sont déployés afin d'assurer une meilleure disponibilité et une tolérance aux pannes. Si l'un des pods devient indisponible, Kubernetes est capable de le recréer automatiquement afin de maintenir le nombre souhaité d'instances.

6.3 Service et exposition de l'application

```yaml
apiVersion: v1
kind: Service
metadata:
  name: devops-app-service
spec:
  type: NodePort
  selector:
    app: devops-app
  ports:
    - port: 80          # Port inside the cluster
      targetPort: 5000  # Port your container exposes
      nodePort: 30080   # Port exposed on your EC2 node
```

FIGURE 6.3.1 – Le fichier Service.yaml

Afin de permettre l'accès à l'application depuis l'extérieur du cluster Kubernetes, un Service est créé. Le Service joue le rôle d'intermédiaire entre les utilisateurs et les pods exécutant l'application. Dans ce projet, un Service de type NodePort est utilisé. Ce type de service permet d'exposer l'application sur un port spécifique de chaque nœud du cluster. Ainsi, l'application devient accessible via l'adresse IP du Worker Node combinée avec le port NodePort défini dans la configuration du service. Le Service assure également la répartition du trafic entre les différents pods du Deployment, garantissant ainsi une meilleure répartition de charge et une meilleure performance globale du système.

6.4 Accès à l'application

Une fois le Deployment et le Service correctement configurés et déployés, l'application devient accessible depuis l'extérieur du cluster. L'accès se fait en utilisant l'adresse IP publique du Worker Node ainsi que le port NodePort associé au Service Kubernetes. Cette configuration permet de tester l'application dans un environnement réel et de vérifier son bon fonctionnement après déploiement. L'utilisateur peut ainsi envoyer des requêtes HTTP à l'application et recevoir des réponses JSON en retour. Grâce à Kubernetes, l'application bénéficie d'une exposition contrôlée, d'une haute disponibilité et d'une gestion automatisée des conteneurs, ce qui garantit un fonctionnement stable et fiable dans un environnement de productio

Chapitre 7

Pipeline CI/CD avec GitHub Actions

7.1 Présentation de GitHub Actions

Afin d'automatiser le processus de développement et de déploiement de l'application, un pipeline CI/CD a été mis en place à l'aide de GitHub Actions. GitHub Actions est un outil d'intégration et de déploiement continu intégré à la plateforme GitHub, permettant d'exécuter automatiquement des workflows en réponse à des événements tels que les commits ou les pushs de code. Dans ce projet, GitHub Actions est utilisé pour automatiser l'ensemble du cycle de vie de l'application, depuis l'intégration du code jusqu'à son déploiement sur le cluster Kubernetes. Cette automatisation permet de réduire les interventions manuelles, d'améliorer la fiabilité du processus et d'accélérer les cycles de livraison.

7.2 Étapes du pipeline

```yaml
name: CI/CD DevOps Pipeline

on:
  push:
    branches: [main]

jobs:

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Installer dependances
        run: pip install -r requirements.txt

      - name: Lancer les tests
```

FIGURE 7.2.1 – Le fichier CI-CD.yaml

Le pipeline CI/CD mis en place est déclenché automatiquement lors de chaque envoi de code sur la branche principale du dépôt GitHub. Il est composé de plusieurs étapes successives qui s'exécutent de manière ordonnée. La première étape consiste à récupérer le code source du projet depuis le dépôt GitHub. Ensuite, les dépendances nécessaires au fonctionnement de l'application sont installées dans l'environnement d'exécution du pipeline. Cette étape permet de préparer l'environnement pour les tests et la construction de l'application. Les étapes suivantes du pipeline incluent l'exécution des tests, la construction de l'image Docker, ainsi que le déploiement de l'application sur Kubernetes. Chaque étape dépend de la réussite de la précédente, garantissant ainsi un enchaînement logique et sécurisé des opérations.

7.3 Tests, build et déploiement automatique

Une étape importante du pipeline consiste à exécuter les tests automatiques de l'application à l'aide de l'outil pytest. Ces tests permettent de vérifier que les fonctionnalités de l'application fonctionnent correctement avant toute mise en production. Si les tests échouent, le pipeline est interrompu et le processus de déploiement n'est pas poursuivi. Si les tests sont validés avec succès, le pipeline passe à l'étape de construction de l'image Docker. L'image de l'application est générée automatiquement à partir du Dockerfile, puis elle est taguée et envoyée vers DockerHub, qui sert de registre central pour le stockage des images Docker. Enfin, la dernière étape du pipeline consiste à déployer l'application sur le cluster Kubernetes. Cela est réalisé en appliquant les fichiers de configuration YAML (manifests), qui définissent les ressources telles que le Deployment et le Service. Cette étape permet de mettre à jour ou de créer les ressources nécessaires au fonctionnement de l'application dans le cluster.

7.4 Résultat du pipeline

Grâce à la mise en place de ce pipeline CI/CD, l'ensemble du processus de développement, de test, de construction et de déploiement est entièrement automatisé. Chaque modification apportée au code source déclenche automatiquement l'exécution du pipeline, garantissant ainsi une intégration continue et un déploiement rapide. Ce mécanisme permet d'améliorer la qualité du code grâce à l'exécution systématique des tests, de réduire les erreurs humaines et d'accélérer la livraison des nouvelles versions de l'application. Le pipeline constitue ainsi un élément central dans l'approche DevOps adoptée dans ce projet, en assurant une automatisation complète et fiable du cycle de vie de l'application.s

.

Chapitre 8

Résultats

8.1 Résultats obtenus et vérification du système

À la fin de ce projet, une infrastructure DevOps complète a été mise en place. L'ensemble du processus de déploiement est automatisé depuis la création de l'infrastructure jusqu'à l'exécution de l'application sur Kubernetes.

Terraform permet de créer rapidement l'infrastructure cloud, tandis qu'Ansible assure la configuration automatique des serveurs. Docker garantit la portabilité de l'application, et Kubernetes permet de gérer efficacement l'exécution des conteneurs.

Le pipeline CI/CD assure quant à lui l'intégration continue et le déploiement automatique de l'application. Chaque modification du code peut être testée et déployée rapidement sans intervention manuelle.

Cette approche permet d'améliorer considérablement la rapidité de développement, la fiabilité du système et la reproductibilité de l'infrastructure.

Conclusion

Ce projet a permis de mettre en pratique plusieurs concepts fondamentaux du DevOps et du Cloud Computing, en s'appuyant sur une chaîne complète d'outils et de technologies modernes. L'utilisation conjointe de Terraform, Ansible, Docker et Kubernetes a rendu possible la mise en place d'une infrastructure cloud automatisée, reproductible et facilement maintenable. Terraform a joué un rôle clé dans la provision des ressources cloud, tandis qu'Ansible a permis d'automatiser la configuration des serveurs et l'installation des dépendances nécessaires au fonctionnement du système. Docker, quant à lui, a assuré la conteneurisation de l'application, garantissant sa portabilité entre différents environnements.
Par ailleurs, Kubernetes a été utilisé pour orchestrer les conteneurs et assurer la gestion dynamique des applications, notamment en termes de déploiement, de mise à l'échelle et de tolérance aux pannes. L'intégration de ces outils dans une architecture cohérente a permis de construire un système complet capable de répondre aux exigences des environnements modernes, où la rapidité de déploiement et la fiabilité sont des critères essentiels.
L'automatisation mise en place tout au long du projet constitue un élément central de l'approche DevOps. Elle permet de réduire significativement les interventions manuelles, limitant ainsi les erreurs humaines et améliorant la cohérence des déploiements. Grâce à cette automatisation, les différentes étapes du cycle de vie de l'application — depuis le provisionnement de l'infrastructure jusqu'au déploiement de l'application — sont exécutées de manière fluide et contrôlée.
De plus, l'intégration d'un pipeline CI/CD basé sur GitHub Actions a permis d'instaurer un processus de développement continu. Chaque modification du code déclenche automatiquement des étapes de test, de construction et de déploiement, garantissant ainsi une meilleure qualité logicielle et une livraison plus rapide des nouvelles versions. Ce mécanisme favorise également une meilleure collaboration entre les équipes et une détection précoce des erreurs.
Enfin, les technologies utilisées dans ce projet sont aujourd'hui largement adoptées dans les environnements professionnels pour la conception de systèmes distribués, scalables et résilients. La maîtrise de ces outils constitue ainsi une compétence essentielle dans les domaines du DevOps et de l'ingénierie cloud, car elle permet de concevoir, automatiser et gérer efficacement des infrastructures modernes adaptées aux besoins des entreprises.